SQL ASSIGNMENT 9

Subject: Database Management Systems (SQL) **Topic:** Advanced Joins (FULL JOIN, SELF JOIN & Optimization) **Status:** Completed

QUESTION 1

Create `Influencers` and `Collaborations` tables. Write a `FULL OUTER JOIN` query to list all influencers and their collaboration partner names, including those with no collaborations.

```
-- Table Schema Creation CREATE TABLE Influencers ( id INT PRIMARY KEY, name VARCHAR(100) ); CREATE TABLE Collaborations ( id INT PRIMARY KEY, influencer1_id INT, influencer2_id INT, collab_date DATE ); -- FULL OUTER JOIN Query SELECT i1.name AS influencer_name, i2.name AS partner_name, c.collab_date FROM Influencers i1 FULL OUTER JOIN Collaborations c ON i1.id = c.influencer1_id FULL OUTER JOIN Influencers i2 ON c.influencer2_id = i2.id;
```

QUESTION 2

Using a `SELF JOIN` on `Playlists`, display each playlist alongside its parent playlist name (for nested Spotify-style playlists).

```
SELECT p.playlist_name AS sub_playlist, parent.playlist_name AS parent_playlist FROM Playlists p LEFT JOIN Playlists parent ON p.parent_playlist_id = parent.id;
```

QUESTION 3

Write a multi-table `LEFT JOIN` query across `Users`, `Orders`, and `Payments` to display each username, order date, and amount, showing all users even if they have no orders/payments.

```
SELECT u.username, o.order_date, p.amount FROM Users u LEFT JOIN Orders o ON u.id = o.user_id LEFT JOIN Payments p ON o.id = p.order_id;
```

QUESTION 4

Eliminate duplicate restaurant rows returned by joining Zomato's `Restaurants` and `Reviews` tables, and explain why duplicates occurred.

Reason for Duplicates: A 1-to-many relationship exists between restaurants and reviews; each review row matches the restaurant ID, causing the restaurant details to repeat for every review written.

```
SELECT DISTINCT r.id, r.name, r.city FROM Restaurants r INNER JOIN Reviews rev ON r.id = rev.restaurant_id;
```

QUESTION 5

Write two different JOIN queries linking `Products` and `Categories` using different join conditions, and explain which condition is more efficient.

```
-- Query 1: Primary Key to Foreign Key Join (Standard & Efficient) SELECT p.product_name, c.category_name FROM Products p INNER JOIN Categories c ON p.category_id = c.id; -- Query 2: Joining on Text Names (In-efficient Alternative) SELECT p.product_name, c.category_name FROM Products p INNER JOIN Categories c ON p.category_name = c.category_name;
```

Efficiency Explanation: Query 1 is significantly faster because it joins indexed numeric integer keys (`category_id = id`), minimizing memory allocation and index search overhead compared to full-string comparisons in Query 2.